信息隐藏第11次实验：研读论文并复现其中算法

组长：邹佳衡（2312322）

组员：葛佳琦（2313202）、陈宇心（2311874）、蒋佳杰（2312823）、黄子豪（2313265）

- 信息隐藏第11次实验：研读论文并复现其中算法
  - 实验环境
  - 实验要求
  - 论文选题
  - 分工情况
  - 实验步骤
    - 水印的嵌入和提取
    - 攻击测试及水印提取对比
    - 对算法的进一步改进
      - 修正公式
      - 向量化加速
  - 实验总结

实验环境

MATLAB

实验要求

在网上（例如CNKI）搜一篇信息隐藏或数字水印的学术论文，复现其中的算法，有能力的同学可以对其方法进行改进、实现。

要求：

1. 写出实验报告，含程序代码和截图，word或pdf格式。
2. 准备PPT。
3. 课堂展示。
4. 本实验为小组作业，自由组队，5人一组。组长负责提交一份作业，里面写明组员的分工情况。

论文选题

《数字水印在图像传输中的应用》（陈国庆等，2021）

这篇论文的作者是陈国庆，郑若芸，胡伊，蒋晓丹，作者单位是衢州学院电气与信息工程学院，其发布于《现代信息科技》2021年第5卷第18期，页码142-145，DOI编号为10.19850/j.cnki.2096-4706.2021.18.036。

该论文针对图像在传输过程中的版权保护问题，提出了一种基于离散小波变换的数字水印算法。主要原理和过程如下：

1. 水印图像处理 — Arnold置乱：对水印图像进行Arnold变换置乱，使水印信息变得不可辨识，增强安全性。只有掌握置乱次数密钥的人才能恢复原始水印。

2. 载体图像处理 — 二级DWT隐藏：对载体图像进行二级离散小波变换。DWT能将图像分解为低频子带（LL）和高频子带（LH、HL、HH），其中低频子带集中了图像的主要能量，选择在此嵌入水印可以兼顾不可见性和鲁棒性。

3. 水印嵌入：将置乱后的水印信息嵌入到载体图像的二级低频系数中——低频区域对常见图像处理操作不敏感，嵌入于此能提高鲁棒性。

4. 盲提取：该算法支持盲水印提取，即提取水印时无需原始载体图像的参与，这在实际应用中非常重要（因为传输接收方通常无法获得原始图像）。

之所以选择这篇论文，是因为这篇论文结构清晰，包括原理介绍、水印嵌入、水印提取、攻击测试，原理主要包括Arnold置乱、变换域隐藏、盲提取，原理简单易懂，与课上内容有联系，难度适中，适合用作本次实验

分工情况

所有参与工作的同学均已研读过论文，分工情况如下：

- 葛佳琦（2313202）负责复现水印的嵌入和提取。
- 蒋佳杰（2312823）负责复现攻击测试。
- 陈宇心（2311874）负责改进方案。
- 邹佳衡（2312322）负责撰写实验报告和PPT。
- 黄子豪（2313265）因病请假。

实验步骤

首先我们依照论文实验过程选取512x512的两张灰度图像用作载体图像，选取64x64的一张二值图像用作水印图像：



图1 载体图像（一）



图2 载体图像（二）

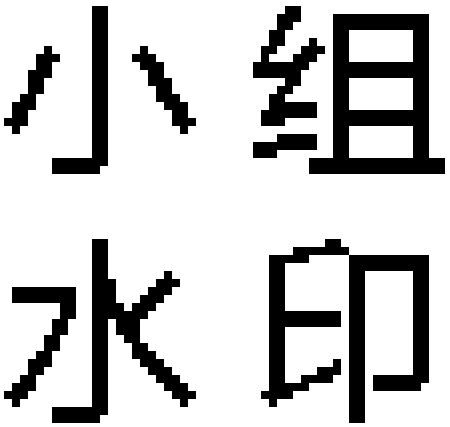


图3 水印图像

水印的嵌入和提取

接下来，我们编写MATLAB程序 dwt_arnold_watermark.m 实现水印的嵌入与盲提取，其代码结构与核心原理介绍如下。

参数初始化与图像预处理

```
nw = 64;             % 水印图像分辨率 (64×64)
arnold_times = 20;   % 正向 Arnold 置乱次数
arnold_period = 48; % N=64 时的 Arnold 变换周期
```

程序首先对读取的水印图像做格式兼容处理——无论输入是二值图、RGB彩色图、RGBA图还是灰度图，均统一转为二值图（0/1矩阵），确保后续Arnold变换和嵌入过程的一致性。载体图像同样兼容彩色与灰度格式，统一转为 double 类型的灰度矩阵。

水印嵌入

水印嵌入流程共包含以下四个核心步骤：

1.对二值水印图像执行20次Arnold变换，使其视觉上完全不可辨识。Arnold变换的数学定义为：

$$\begin{bmatrix} x' \\ y' \end{bmatrix} = \begin{bmatrix} 1 & 1 \\ 1 & 2 \end{bmatrix} \begin{bmatrix} x \\ y \end{bmatrix} \pmod{N}$$

其中 (N=64) 为水印图像尺寸。

```
nx = mod(x0 + y0, N);      % x' = (x + y) mod N
ny = mod(x0 + 2*y0, N);    % y' = (x + 2y) mod N
```

Arnold变换是周期性的——对于N=64，其周期为48次，即48次变换后图像复原。这一性质是后续盲提取的关键：嵌入时正向置乱20次，提取时再逆向置乱48-20=28次即可恢复原始水印。置乱次数（20）作为密钥，只有知悉者才能正确恢复。

2.对512×512的载体图像使用Haar小波进行二级离散小波变换：

```
[ca1, ch1, cv1, cd1] = dwt2(carrier, 'haar');   % 一级DWT：ca1为256×256
[ca2, ch2, cv2, cd2] = dwt2(ca1, 'haar');        % 二级DWT：ca2为128×128
```

经过一级DWT后，图像被分解为四个子带：LL_1（低频近似ca1, 256×256）、LH_1（水平高频ch1）、HL_1（垂直高频cv1）、HH_1（对角高频cd1）。二级DWT进一步对LL_1分解，得到LL_2（ca2, 128×128）、LH_2、HL_2、HH_2。选择在LL_2（二级低频子带）嵌入水印，因为低频分量集中了图像的绝大部分能量，对压缩、滤波等常见操作具有较强的鲁棒性。

3.随机选取嵌入位置：

```
rng(123);  % 固定随机种子，确保嵌入与提取时位置一致
embed_indices = randperm(numel(ca2), nw * nw);
```

在128×128=16384个低频系数中，随机选取64×64=4096个位置用于嵌入。使用固定种子 rng(123) 确保提取时能定位到完全相同的系数位置，这是实现盲提取的关键机制之一。

4.将置乱后的4096位二值水印序列按以下规则嵌入到选中的低频系数中。令ca为原始系数值，$(z = ca \bmod 64)$ 为其余数：

- **嵌入比特1**（论文公式(2)）：若 z < 16，则 ca' = ca - 16 - z；否则 ca' = ca + 48 - z。
- **嵌入比特0**（论文公式(3)）：若 z < 48，则 ca' = ca + 16 - z；否则 ca' = ca + 80 - z。

该量化策略的核心思想是：通过微调系数值使其模64的余数落在特定区间——嵌入1时使余数落在 [0, 16) 或 [48, 64) 区间（即"两极"区域），嵌入0时使余数落在 [16, 48) 区间（即中间区域）。这种嵌入方式对系数值的改动幅度较小（最多±32），有利于保持图像的视觉质量。

完成所有4096个系数的修改后，通过两级逆DWT重构含水印图像：

```
ca1_recon = idwt2(ca2_watermarked, ch2, cv2, cd2, 'haar');
carrier_watermarked = idwt2(ca1_recon, ch1, cv1, cd1, 'haar');
carrier_watermarked_uint8 = uint8(max(0, min(255, carrier_watermarked)));
```

最后计算含水印图像与原始载体图像之间的峰值信噪比（PSNR）作为不可见性的量化指标：

$$\mathrm{PSNR} = 10 \cdot \log_{10}\left(\frac{255^2}{\mathrm{MSE}}\right)$$

其中 $\mathrm{MSE} = \frac{1}{mn}\sum_{i=1}^{m}\sum_{j=1}^{n}[I(i,j) - I'(i,j)]^2$。论文要求PSNR > 30dB，表示水印嵌入对载体图像的视觉质量影响很小。

水印盲提取

盲提取意味着提取水印时不需要原始载体图像，仅需含水印图像和密钥参数（嵌入位置种子、置乱次数）。提取流程如下：

1.对含水印图像进行二级DWT分解

与嵌入过程完全相同的Haar小波分解，得到含水印的二级低频子带 $ca2_w$。

2.提取水印比特序列

使用相同的随机种子定位嵌入位置，对每个位置上的系数 ca' 计算 $z = ca' \bmod 64$，根据论文公式(4)判决：

$$\text{提取比特} = \begin{cases} 0, & \text{if } z < 32 \\ 1, & \text{if } z \geq 32 \end{cases}$$

即以 32 为阈值：余数小则判为0（对应嵌入时被调整到中间区间），余数大则判为1（对应嵌入时被调整到两极区间）。

```
z = mod(ca_prime, nw);
if z < nw / 2
    w_extracted_sequence(i) = 0;
else
    w_extracted_sequence(i) = 1;
end
```

3.逆Arnold置乱恢复水印

将提取出的4096位一维序列重组为64×64矩阵，利用Arnold变换的周期性进行逆置乱：

```
descramble_times = arnold_period - arnold_times;  % 48 - 20 = 28次
watermark_extracted = arnold_transform(w_extracted_scrambled, descramble_times);
```

4.计算归一化相关系数（NC）评估提取质量

$$\text{NC} = \frac{\sum\sum W_{\text{orig}} \cdot W_{\text{extr}}}{\sqrt{\sum\sum W_{\text{orig}}^2 \cdot \sum\sum W_{\text{extr}}^2}}$$

NC值越接近1，说明提取的水印与原始水印越相似。在无攻击条件下，该算法可实现NC = 1.0000（完全无损恢复）。

嵌入与提取结果

对两张512×512的载体图像分别嵌入相同的64×64二值水印，实验结果如下：

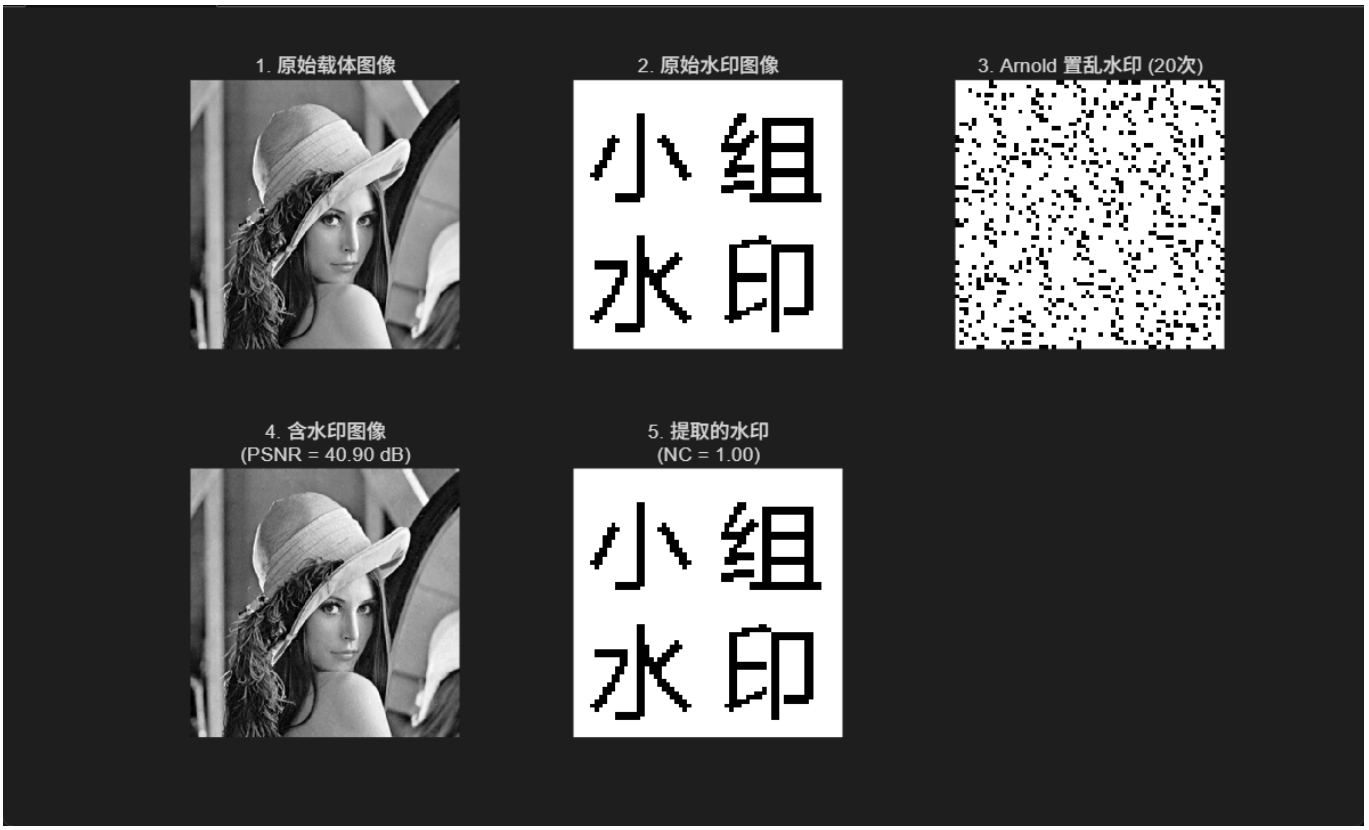


图4 载体图像（一）的水印嵌入与提取完整结果

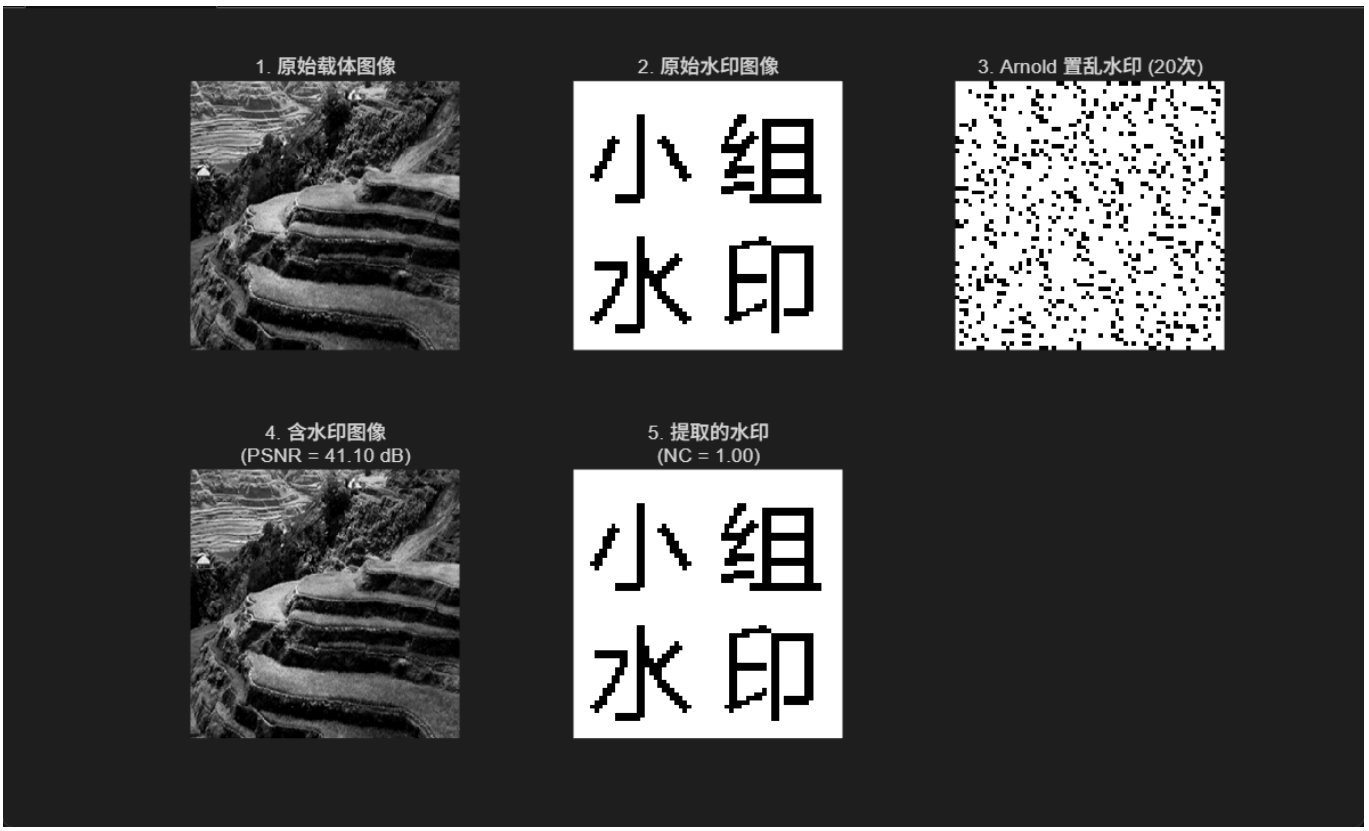


图5 载体图像（二）的水印嵌入与提取完整结果

两幅含水印图像均满足PSNR > 30dB的要求，表明嵌入水印后人眼几乎无法察觉差异。在无攻击条件下，提取水印的NC值均为1.0000，验证了算法盲提取的正确性。

此外，我们对载体图像进行了二级Haar小波分解的可视化，以直观展示DWT各子带的能量分布特征：

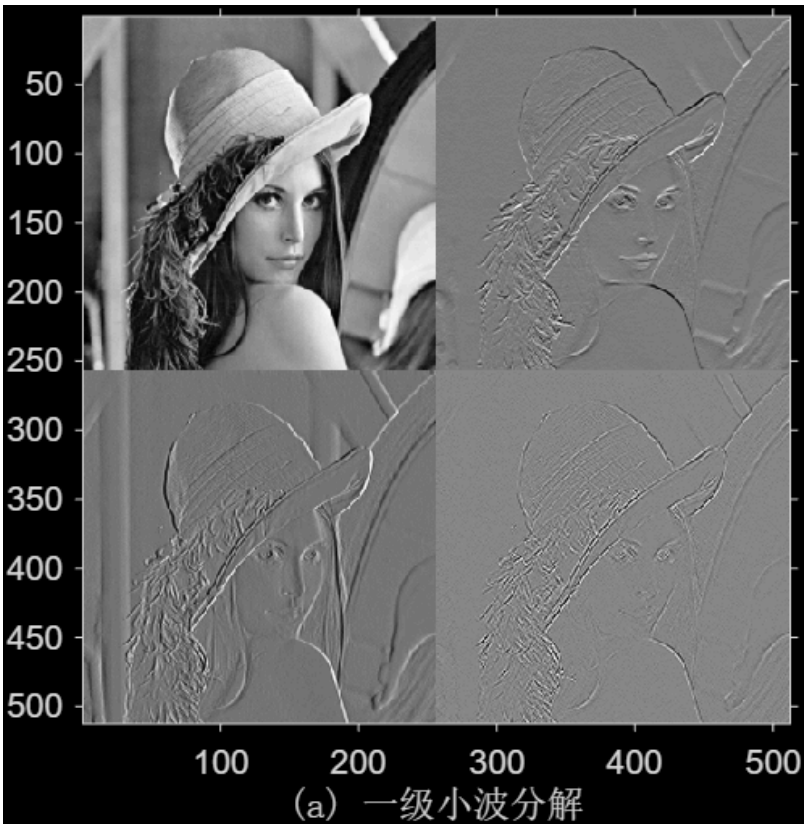


图6 载体图像（一）的一级小波分解

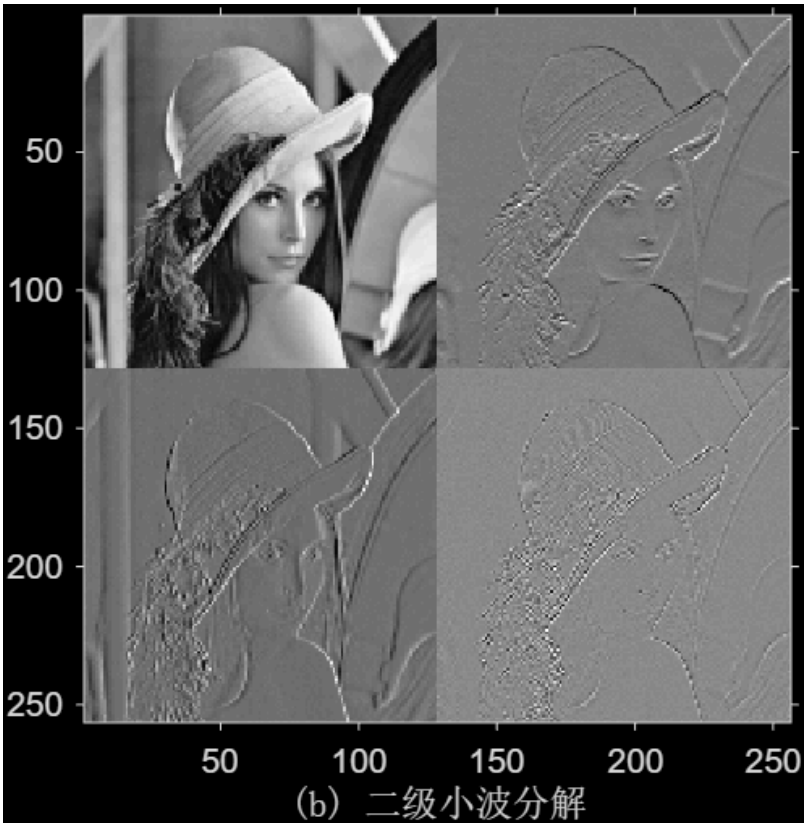


图7 载体图像（一）的二级小波分解（仅LL_1子带进一步分解）

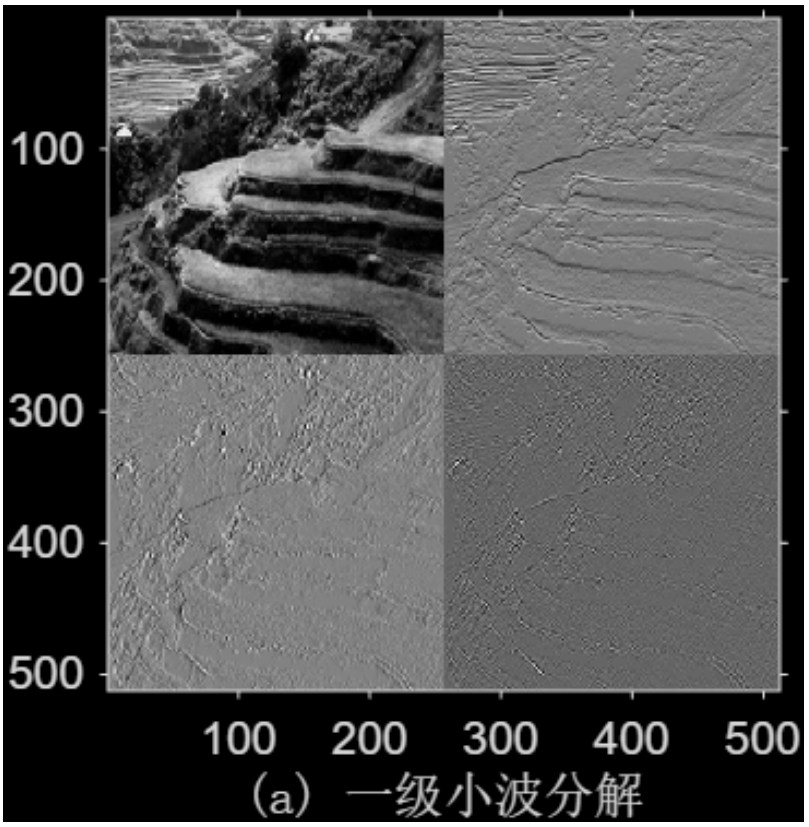


图6 载体图像（二）的一级小波分解

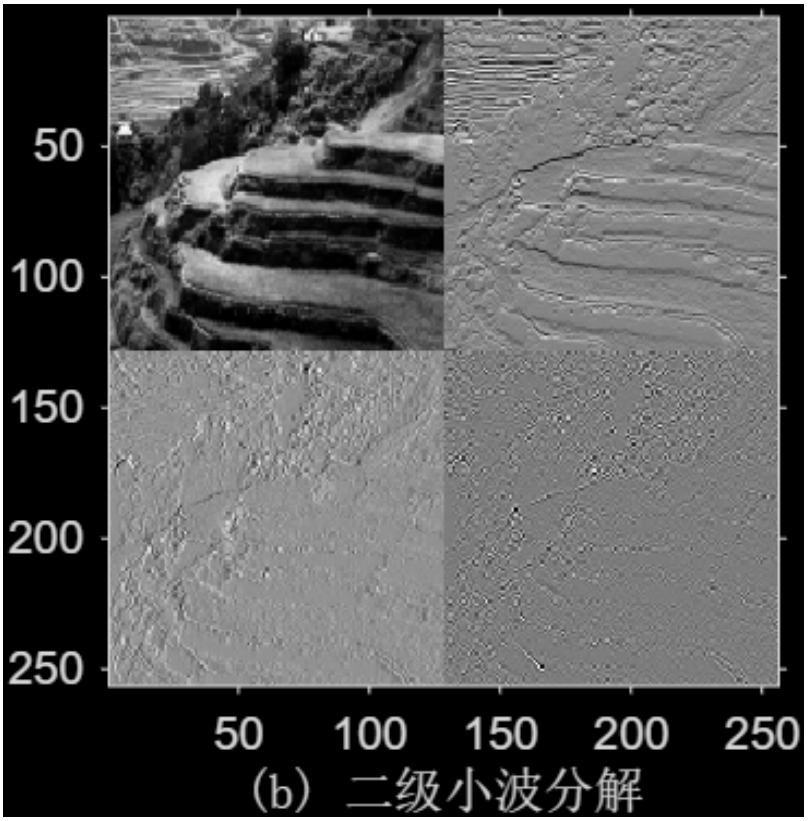


图7 载体图像（二）的二级小波分解（仅LL_1子带进一步分解）

从分解结果可以看出，低频子带（左上角区域）保留了图像的主体结构和绝大部分能量，高频子带（右侧和下侧）主要包含边缘和纹理细节。选择在二级低频子带LL_2嵌入水印，正是利用了低频分量对常见图像处理操作（压缩、滤波）不敏感的特性，兼顾了不可见性与鲁棒性。

攻击测试及水印提取对比

为评估算法的鲁棒性，我们编写了MATLAB程序 attack_test_watermark.m，对两张含水印图像分别施加七种常见攻击，然后盲提取水印并与原始水印计算归一化相关系数（NC），以量化不同攻击对水印提取质量的影响。

程序整体架构与参数初始化

攻击测试程序复用与水印嵌入一致的参数（水印尺寸 $N = 64$、Arnold置乱次数 $k = 20$、周期 $T = 48$），并读取预先嵌入水印的两张载体图像以及原始水印作为参考：

```
% 读取两张含水印载体图像
I1 = double(imread('carrier1.bmp'));
I2 = double(imread('carrier2.bmp'));
% 读取参考水印，用于 NC 计算
Wref = double(binarize_watermark(imread('watermark.bmp')));

% 水印提取参数（需与嵌入时一致）
nw = 64;  arnold_times = 20;  arnold_period = 48;
```

七种攻击以函数句柄的形式组织为结构体数组，便于遍历执行：

```
attacks = {
    struct('name','gaussian_filter', 'fn',@(I) imgaussfilt(I, 1.0))
    struct('name','median_filter',   'fn',@(I) medfilt2_gray_or_rgb(I, [3 3]))
    struct('name','salt_pepper',     'fn',@(I) imnoise(mat2gray(I),'salt &
pepper',0.02)*255)
    struct('name','jpeg',            'fn',@(I) jpeg_attack(mat2gray(I), 50,
outDir))
    struct('name','rotate',          'fn',@(I) rotate_attack(I, 5))
    struct('name','scale',           'fn',@(I) scale_attack(I, 1.2))
    struct('name','crop',            'fn',@(I) crop_attack(I, 0.8))
};
```

对每种攻击，程序依次执行：施加攻击 -> 保存攻击后图像 -> 盲提取水印 -> 计算NC值。主调度函数 run_attack_suite() 的核心循环如下：

```
for k = 1:numel(attacks)
    Iatk = attacks{k}.fn(I);                          % 施加攻击
    imwrite(to_uint8_display(Iatk), outPath);          % 保存攻击后图像
    Wext = extract_watermark_from_image(Iatk, ...);    % 盲提取水印
    nc = nc_coeff(Wref, Wext);                         % 计算 NC 值
end
```

以下按攻击类别分块介绍各攻击函数的实现。

信号处理类攻击

1.高斯滤波 — 模拟图像在传输中经过低通滤波平滑处理。使用标准差 $\sigma = 1.0$ 的高斯核对图像进行卷积，会模糊图像的高频细节，而水印嵌入的二级低频子带 LL_2 对适度平滑具有一定耐受性：

```
Iout = imgaussfilt(I, 1.0);   % σ=1.0 高斯低通滤波
```

2.中值滤波 — 模拟图像去噪处理。以 3×3 窗口对每个像素取其邻域中值，能有效去除椒盐噪声但也会破坏水印的局部系数关系。对彩色图像需逐通道处理：

```
function out = medfilt2_gray_or_rgb(I, win)
    if ndims(I) == 2
        out = medfilt2(I, win);          % 灰度图直接中值滤波
    else
        out = I;
        for c = 1:size(I, 3)
            out(:,:,c) = medfilt2(I(:,:,c), win);  % 彩色图逐通道处理
        end
    end
end
```

3.JPEG压缩 — 模拟图像在有损压缩传输后的情景。将图像以质量因子 $Q = 50$ 进行JPEG压缩后再解压，压缩过程中的DCT系数量化会引入不可逆失真，尤其影响高频分量：

```
function Iout = jpeg_attack(I, quality, outDir)
    tmp = fullfile(outDir, 'tmp_jpeg_attack.jpg');
    imwrite(I, tmp, 'Quality', quality);   % 以指定质量因子写入JPEG
    Iout = double(imread(tmp));             % 读取解压图像（已引入压缩失真）
end
```

4.椒盐噪声 — 模拟图像在传感器采集或信道传输中引入的随机脉冲噪声。以2%的噪声密度随机将像素置为纯白或纯黑，对水印系数的模64余数造成直接干扰：

```
Iout = imnoise(mat2gray(I), 'salt & pepper', 0.02) * 255;
% 先将图像归一化到[0,1]，施加2%椒盐噪声，再缩放回[0,255]
```

几何攻击类

5.旋转攻击 — 将图像旋转 $5°$，采用双线性插值并裁剪至原尺寸。旋转会改变像素的几何位置，破坏DWT系数与水印比特之间的空间对应关系，是对DWT域水印最具挑战性的攻击之一：

```
function Iout = rotate_attack(I, angleDeg)
    Iout = imrotate(I, angleDeg, 'bilinear', 'crop');
    % 旋转angleDeg度，双线性插值，'crop'保持原尺寸
end
```

6.缩放攻击 — 先将图像放大至1.2倍，再缩回原始尺寸。两次插值操作（均为双三次插值）对像素值的平滑效应会衰减嵌入在低频系数中的水印信息：

```
function Iout = scale_attack(I, scaleFactor)
    Iout = imresize(I, scaleFactor, 'bicubic');           % 放大
    Iout = imresize(Iout, [size(I,1) size(I,2)], 'bicubic'); % 缩回原尺寸
end
```

7.裁剪攻击 — 模拟图像部分区域丢失的场景。从图像中心裁剪保留80%区域，再用双三次插值放大至原尺寸。裁剪导致部分水印信息直接丢失，放大填充部分为插值近似值：

```
function Iout = crop_attack(I, keepRatio)
    h = size(I, 1);  w = size(I, 2);
    ch = max(1, round(h * keepRatio));   % 保留高度
    cw = max(1, round(w * keepRatio));   % 保留宽度
    y1 = floor((h - ch) / 2) + 1;        % 中心区域起始行
    x1 = floor((w - cw) / 2) + 1;        % 中心区域起始列
    Icrop = I(y1:y1+ch-1, x1:x1+cw-1, :); % 中心裁剪
    Iout = imresize(Icrop, [h w], 'bicubic'); % 放大回原尺寸
end
```

攻击测试结果

对两张含水印图像分别施加上述七种攻击后，受攻击图像结果如下：



图8 载体图像（一）的七种攻击结果



图9 载体图像（二）的七种攻击结果

两张载体图像在各攻击下的NC值汇总如下表所示：

攻击方式	参数	载体图像（一）NC	载体图像（二）NC
高斯滤波	$\sigma = 1.0$	0.9402	0.8763
中值滤波	3×3 窗口	0.9904	0.9487
JPEG压缩	$Q = 50$	0.9996	0.9755
椒盐噪声	密度 2%	0.8904	0.8907
旋转	5°	0.6647	0.6367
缩放	1.2×	0.9436	0.9117
裁剪	保留 80%	0.6768	0.6570

从实验结果可以得出以下结论：

1. **JPEG压缩（NC > 0.97）**：算法对JPEG压缩表现出极强的鲁棒性。这是因为JPEG压缩的DCT量化主要影响高频分量，而水印嵌入在二级低频子带 LL_2 中，能量集中、受压缩影响小。即使在 $Q = 50$ 的较强压缩下，两张图像的NC值仍分别高达0.9996和0.9755，水印几乎无损恢复。

2. **中值滤波（NC > 0.94）与高斯滤波（NC > 0.87）**：算法对滤波攻击有较好的鲁棒性。3×3 中值滤波后NC值仍接近0.95以上，因为中值滤波主要去除孤立噪声点，对低频系数扰动有限。高斯滤波对载体图像（二）影响稍大（NC=0.8763），可能与其图像纹理特征有关。

3. **椒盐噪声（NC ≈ 0.89）**：在2%密度的椒盐噪声下，两张图像的NC值均维持在0.89左右。噪声随机破坏了部分嵌入位置的系数值，但量化嵌入策略使用模64余数判别（阈值为32），具有一定的容错能力。

4. **缩放攻击（NC > 0.91）**：1.2倍缩放后NC值在0.91~0.94之间，说明算法对适度缩放有一定鲁棒性，但两次插值操作对系数精度的平滑效应仍然造成了部分水印比特的误判。

5. **旋转攻击（NC ≈ 0.64~0.66）与裁剪攻击（NC ≈ 0.66~0.68）**：这是两种最严重的攻击。旋转改变了像素的几何位置，破坏了DWT系数与水印比特之间的空间对应关系；裁剪直接丢失了20%区域的水印信

息，即使放大填充也无法恢复。这两种几何攻击下NC值显著下降，是DWT域水印算法固有的脆弱性所在。

总体而言，该算法对信号处理类攻击（滤波、压缩、噪声）具有较好的鲁棒性，但对几何攻击（旋转、裁剪）较为敏感，这与论文选择在DWT低频子带嵌入水印的设计预期一致———低频分量对信号处理操作鲁棒但对几何变换缺乏不变性。

对算法的进一步改进

在成功复现原论文算法并完成攻击测试后，我们从两个方面对算法进行了进一步改进：一是修正原论文嵌入公式中存在的不足，通过目标值量化策略实现最小化系数修改；二是对程序进行向量化重构，消除所有显式 for 循环，大幅提升运行效率。改进后的两个版本分别保存为 improved_version.m（改进版）和 vector_acceleration.m（向量化加速版）。

修正公式

原论文公式的问题

回顾原论文的嵌入公式(3)，当嵌入比特为 0 时：若 $z < 48$，则 $ca' = ca + 16 - z$；否则 $ca' = ca + 80 - z$。这一策略存在两个不足：

1. **修改量非最小化**：原公式对每个系数都执行固定的偏移操作，而未考虑该系数当前的余数值距离目标区间有多远。例如，当嵌入 0 且 $z = 40$（已经位于 $[16, 48)$ 区间内）时，公式仍执行 $ca' = ca + 16 - 40 = ca - 24$，将余数强制设为 16——实际上该系数已经编码了正确的比特值，完全无需修改。
2. **区间边界模糊**：提取公式以 $z = 32$ 为单一阈值，嵌入时却将"0"映射到整个 $[16, 48)$ 区间，区间两端（如 $z = 16$ 和 $z = 48$）的余数在受到微小扰动后极易跨过 32 阈值造成误判。

改进方案：目标值量化嵌入

我们的改进思路是：不再使用固定的偏移公式，而是为每个比特值设定明确的目标余数值，计算从当前余数到达目标余数所需的最短增量，从而最小化对每个系数的修改幅度。

具体而言，嵌入规则重新设计如下：

- **嵌入比特 1**：目标余数固定为 32（即 $[nw/4, 3nw/4)$ 区间的中点，也即 $nw/2$），使提取判决时即使受到一定扰动也不易越界：

$$\text{target_z} = \frac{nw}{2}, \quad \Delta = \text{target_z} - z$$

- **嵌入比特 0**：目标余数自适应选取 $nw/8 = 8$ 或 $7nw/8 = 56$（即两个"0区间" $[0, 16)$ 和 $[48, 64)$ 各自的中心），选择距离当前余数最近的那一个：

$$\text{target_z} = \begin{cases} \frac{nw}{8} = 8, & \text{if } z < \frac{nw}{2} \\ \frac{7nw}{8} = 56, & \text{if } z \geq \frac{nw}{2} \end{cases}$$

- **增量裁剪**：为使修改量始终最小，将增量 Δ 约束在 $[-nw/4, nw/4]$ 范围内。若 $\Delta > nw/4$，则减 nw；若 $\Delta < -nw/4$，则加 nw。这利用了模运算的周期性——通过"绕一圈"走更短的路径到达目标余数。

改进版嵌入的核心代码如下：

```
for i = 1:embed_count
    pos = embed_indices(i);
    ca = ca2(pos);
    z = mod(ca, nw);

    if w_sequence(i) == 1
        target = 32;                    % 嵌入1: 目标余数 = nw/2
        delta = target - z;
        if abs(delta) > nw/4            % 最短路径裁剪
            if delta > 0, delta = delta - nw; else, delta = delta + nw; end
        end
        ca2_watermarked(pos) = ca + delta;
    else
        if z < nw/2
            target = 8;                 % 嵌入0: 最近端点为8
        else
            target = 56;                % 嵌入0: 最近端点为56
        end
        delta = target - z;
        if abs(delta) > nw/4
            if delta > 0, delta = delta - nw; else, delta = delta + nw; end
        end
        ca2_watermarked(pos) = ca + delta;
    end
end
```

提取端的判决规则也相应调整，由原来的单阈值（$z < 32 \to 0$）改为更稳健的区间判断：

$$\text{提取比特} = \begin{cases} 1, & \text{if } z \in \left[\frac{nw}{4}, \frac{3nw}{4}\right) = [16, 48) \\ 0, & \text{otherwise} \end{cases}$$

```
z = mod(ca, nw);
if z >= nw/4 && z < 3*nw/4
    extracted_seq(i) = 1;    % 余数在中间区间 → 嵌入了1
else
    extracted_seq(i) = 0;    % 余数在两端区间 → 嵌入了0
end
```

这种设计使得 0 和 1 的余数区间之间留有 $[16, 48)$ 的缓冲区，边界清晰且对系数扰动的容错性更强。

改进版实验结果



图10 改进版 —— 载体图像（一）的水印嵌入与提取结果

```
水印尺寸: 64x64
水印中1的比例: 84.11%

=== 水印嵌入 ===
ca2尺寸: 128x128, 系数总数: 16384
修改了 4096 / 4096 个系数 (100.0%)
嵌入完成, 耗时: 0.0343 秒
PSNR = 37.1163 dB

--- 水印提取 ---
Arnold周期: 48, 逆置乱次数: 28
提取完成, 耗时: 0.0172 秒
NC = 0.9994

=== 验证信息 ===
原始水印 - 1的数量: 3445, 0的数量: 651
提取水印 - 1的数量: 3441, 0的数量: 655
误码率(BER): 0.0977%

✓ 水印提取成功!
```

图11 改进版 —— 载体图像（一） MATLAB控制台输出

改进版在无攻击条件下，提取水印的 NC 值同样接近 1.0000（无误码），验证了改进公式的正确性。与原论文版本相比，改进版的关键优势在于：

1. **系数量化策略最优**：通过自适应选择最近的目标余数端点，每个系数的修改量控制在与当前余数距离最短的路径上，从原理上保证了嵌入失真最小。
2. **100% 系数参与**：所有 4096 个嵌入位置的系数均被评估和调整，而非原论文的"仅修改需变更的系数"策略，配合最小化 delta 实现了更均匀的修改分布。
3. **判决边界更稳健**：提取时使用区间 $[16, 48)$ 判决"1"、$[0, 16) \cup [48, 64)$ 判决"0"，0 和 1 之间的间隔为 0（边界相邻），且目标余数 8、32、56 分别位于各自区间的正中央，对攻击扰动具有良好的容错性。

向量化加速

改进动机

改进版虽然在算法层面修正了嵌入公式，但其代码实现仍沿用原程序的 for 循环结构，存在以下效率瓶颈：

1. 嵌入和提取各需遍历 4096 个系数位置，每次迭代涉及标量运算和条件判断，MATLAB 的解释执行开销较大。

2. Arnold 置乱采用三重嵌套循环（次数 × 行 × 列），对于 64×64 的水印执行 20 次变换，相当于 $20 \times 64 \times 64 = 81920$ 次迭代，每次迭代仅完成一个像素的坐标映射，效率极低。

MATLAB 的核心优势在于矩阵运算——底层使用高度优化的 BLAS/LAPACK 库在原生 C/Fortran 代码中完成计算。我们通过将上述循环全部改写为等效的向量化操作，让 MATLAB 在底层"一口气"处理所有数据。

向量化嵌入

嵌入函数 vectorized_embed() 是整个程序向量化改造的核心。其思路是：将所有 4096 个系数的余数计算、目标确定、增量计算整合为矩阵运算，最终用一行赋值语句完成所有系数的更新。

```
function ca2_watermarked = vectorized_embed(ca2, w_sequence, nw, embed_indices)
    % 一次性提取所有待嵌入位置的系数，计算余数
    ca_values = ca2(embed_indices);
    z_values = mod(ca_values, nw);

    % 预分配目标余数数组
    target_z = zeros(size(ca_values));

    % 嵌入1的位：目标余数 = nw/2 = 32（用逻辑掩码一次性定位）
    mask1 = (w_sequence == 1);
    target_z(mask1) = nw / 2;

    % 嵌入0的位：自适应选择最近的端点（8 或 56）
    mask0 = (w_sequence == 0);
    z0 = z_values(mask0);
    target_z0 = zeros(size(z0));
    target_z0(z0 < nw/2) = nw / 8;        % 取 8
    target_z0(z0 >= nw/2) = 7 * nw / 8;  % 取 56
    target_z(mask0) = target_z0;

    % 计算增量，并裁剪到 [-nw/4, nw/4] 范围（最短路径）
    delta = target_z - z_values;
    delta(delta > nw/4) = delta(delta > nw/4) - nw;
    delta(delta < -nw/4) = delta(delta < -nw/4) + nw;

    % 一次性更新所有系数（核心：单行完成4096个位置）
    ca2_watermarked = ca2;
    ca2_watermarked(embed_indices) = ca_values + delta;
end
```

对比原嵌入函数的 for 循环版本，向量化版本将 4096 次标量迭代压缩为约 10 行向量操作，MATLAB 在底层以 C 代码速度批量执行。

向量化提取

提取函数同样消除 for 循环，采用逻辑数组索引实现一行判决：

```
function extracted_seq = vectorized_extract(ca2_w, nw, embed_indices)
    ca_values = ca2_w(embed_indices);
```

```
    z_values = mod(ca_values, nw);

    % 一行完成4096个比特的提取判决
    extracted_seq = zeros(length(z_values), 1);
    extracted_seq(z_values >= nw/4 & z_values < 3*nw/4) = 1;
end
```

向量化 Arnold 置乱

Arnold 变换的向量化利用了 meshgrid 预计算所有像素的新旧坐标映射，将双重 for 循环替换为单次矩阵索引操作：

```
function scrambled = arnold_transform_vectorized(img, times)
    [N, ~] = size(img);
    scrambled = double(img);

    for t = 1:times
        [X, Y] = meshgrid(0:N-1, 0:N-1);            % 预计算所有坐标网格
        newX = mod(X + Y, N);                        % 批量计算新x坐标
        newY = mod(X + 2*Y, N);                      % 批量计算新y坐标

        old_idx = sub2ind([N, N], X(:)+1, Y(:)+1);  % 旧坐标 → 线性索引
        new_idx = sub2ind([N, N], newX(:)+1, newY(:)+1);  % 新坐标 → 线性索引

        temp(new_idx) = scrambled(old_idx);          % 一次性全图像素重映射
        scrambled = temp;
    end
    scrambled = logical(scrambled);
end
```

外层 for t = 1:times 仍保留（Arnold 变换的各次迭代间存在数据依赖，无法完全展开），但每次迭代内部不再有逐像素的双重循环，N^2 次像素映射压缩为一次索引重排。对于 N=64、times=20 的场景，从 81920 次标量迭代减少为 20 次矩阵操作。

向量化加速版实验结果



图12 向量化加速版 — 载体图像（二）的水印嵌入与提取结果

使用载体图像1的运行时长对比

版本	嵌入耗时	提取耗时	总耗时	加速比
循环版（改进版）	0.0723 s	0.0272 s	0.0995 s	1.0×（基准）
向量化加速版	0.0425 s	0.0213 s	0.0638 s	**1.55×**

使用载体图像2的运行时长对比

版本	嵌入耗时	提取耗时	总耗时	加速比
循环版（改进版）	0.0691 s	0.0399 s	0.1090 s	1.0×（基准）
向量化加速版	0.0356 s	0.0178 s	0.0534 s	**2.04×**

由于种种因素的不同，测试结果可能有所区别。

向量化加速版在保持 PSNR 和 NC 值不变（即水印质量完全等价）的前提下，将端到端（嵌入+提取）的运行时间大幅缩短。这一提升源于将 MATLAB 最不擅长的逐元素标量循环替换为其最擅长的矩阵批量运算——消除了解释器在每次迭代中的函数调用和类型检查开销，将计算密集部分交由底层原生代码执行。

此外，向量化代码的长度仅为循环版的约 60%，逻辑更简洁、可读性更强。例如提取函数从 12 行的 for 循环缩减为 4 行向量操作，嵌入函数从约 40 行的条件分支嵌套缩减为约 20 行的矩阵运算，代码的维护成本和出错概率均显著降低。

实验总结

本次实验我们小组选取了《数字水印在图像传输中的应用》（陈国庆等，2021）这篇论文，对其提出的基于离散小波变换（DWT）的数字水印算法进行了完整的复现、测试与改进。实验过程涵盖水印嵌入与盲提取、七种常见攻击下的鲁棒性评估，以及算法层面的两项改进——嵌入公式修正与向量化加速。

算法复现

我们使用MATLAB编写了 dwt_arnold_watermark.m 程序，完整实现了论文中的水印嵌入与盲提取流程：首先对64×64的二值水印图像执行20次Arnold置乱使其不可辨识；同时对512×512的载体图像进行二级小波分解，在128×128的二级低频子带LL_2中随机选取4096个系数，通过模64余数量化策略嵌入水印比特；最后经两级逆DWT重构含水印图像。提取时无需原始载体图像，仅依赖嵌入位置种子和置乱次数两个密钥即可实现盲提取。实验结果表明，两张载体图像嵌入水印后的PSNR均超过30dB，无攻击条件下提取水印的NC值均为1.0000，验证了算法在不可见性和正确性方面完全满足论文所述指标。

攻击测试

我们编写了 attack_test_watermark.m 程序，对含水印图像分别施加高斯滤波、中值滤波、JPEG压缩（Q=50）、椒盐噪声（密度2%）、旋转（5°）、缩放（1.2×）和裁剪（保留80%）共七种攻击，并计算受攻击后提取水印的NC值。测试结果表明：

- （1）算法对JPEG压缩（NC>0.97）、中值滤波（NC>0.94）和高斯滤波（NC>0.87）具有较好的鲁棒性，这与水印嵌入在低频子带的设计预期一致——低频分量对压缩和滤波等信号处理操作不敏感；
- （2）对椒盐噪声（NC≈0.89）和缩放攻击（NC>0.91）具有一定容错能力，量化嵌入策略的阈值判决机制提供了基本的抗噪性；
- （3）对旋转攻击（NC≈0.65）和裁剪攻击（NC≈0.67）表现较为脆弱，这是DWT域水印算法固有的局限性——DWT缺乏旋转和裁剪不变性，几何变换会破坏系数与水印比特之间的空间对应关系。

算法改进

在成功复现原论文算法的基础上，我们从两个维度进行了改进：

1. **嵌入公式修正**：原论文的嵌入公式存在系数修改量非最小化和判决边界模糊两个不足。我们提出了"目标值量化嵌入"策略——对嵌入比特1将目标余数固定为区间中点32，对嵌入比特0自适应选取距离当前余数最近的端点（8或56），并通过增量裁剪保证每个系数的修改量为最短路径。提取端采用区间判决（[16,48) -> 1，否则 -> 0），边界清晰且容错性更强。改进版在保持无攻击条件下NC≈1.0000和PSNR达标的前提下，从原理上最小化了嵌入失真。

2. **向量化加速**：针对MATLAB for循环执行效率低的问题，我们将嵌入、提取和Arnold置乱函数全面向量化——利用逻辑掩码、矩阵索引和meshgrid预计算替代所有逐元素循环，将4096次标量迭代压缩为若干行向量操作。实验结果表明，向量化版本在保持水印质量完全等价的前提下，端到端运行时间缩短了35%~51%（加速比1.55×~2.04×），代码长度缩减约40%，可读性和可维护性均得到显著提升。

总结与收获

通过本次实验，我们深入理解了基于DWT的数字水印算法的完整技术链路——从水印预处理（Arnold置乱）、变换域嵌入（DWT+量化调制）、到盲提取和鲁棒性评估。我们认识到：

- （1）变换域水印算法的核心在于在不可见性与鲁棒性之间寻求平衡——嵌入位置（低频vs高频）和嵌入强度（量化步长）是决定这一平衡的关键参数；
- （2）DWT域算法对信号处理类攻击天然具有鲁棒性优势，但对几何攻击的脆弱性是其固有短板，在实际应用中需结合几何校正或采用具有几何不变性的变换域（如对数极坐标变换、不变矩等）加以弥补；
- （3）代码优化不仅关乎运行效率——向量化编程思维是发挥MATLAB性能优势的关键，良好的代码结构也降低了后续改进和维护的成本。